

Classical Signal Processing: Finding Peaks

Wigner Summer Camp

Data and Compute Intensive Sciences Research Group

Balázs, Paszkál, Vince, Levente, Bence, Antal
Éva, Hajni

6–10 July 2026



Contents

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

What is a Peak?

- ▶ A **peak** in a discrete signal x is an index p satisfying

$$x[p-1] < x[p] \quad \text{and} \quad x[p] > x[p+1]. \quad (1)$$

- ▶ In other words, a peak is a *strict local maximum*.
- ▶ Peak finding is a fundamental step in many signal processing pipelines.
- ▶ In anomaly detection, anomalies often manifest as unusually large or sharp peaks in a derived feature signal.

scipy.signal.find_peaks()

```
1 from scipy.signal import find_peaks
2
3 peaks, properties = find_peaks(x, # Data
4     height=None,          # filter by peak value
5     threshold=None,       # filter by drop to
6         neighbors
7     distance=None,        # minimum separation
8     prominence=None,      # filter by prominence
9     width=None,           # filter by width
10    wlen=None,             # window for
11        prominence/width
12    rel_height=0.5,        # level for width
13        measurement
14    plateau_size=None      # filter by flat-top size
15 )
```

- ▶ `peaks` is an array of indices of accepted peaks.
- ▶ `properties` is a dictionary of computed peak attributes.
- ▶ [Documentation](#).

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

The `height` Parameter

- ▶ Filters peaks by the absolute value of the signal at the peak, $x[p]$.
- ▶ Accepts a scalar h (minimum height) or a pair $(h_{\min}, h_{\max})$:

$$h_{\min} \leq x[p] \leq h_{\max}. \quad (2)$$

- ▶ **None** in either position means that bound is not applied.
- ▶ The accepted peak heights are stored in `properties['peak_heights']`.

The height Parameter – Example

Example 1 (height filtering)

```
1 import numpy as np
2 from scipy.signal import find_peaks
3
4 x = np.array([1, 3, 1, 4, 1, 2, 1])
5 # Peaks at indices 1 (value 3), 3 (value 4), 5
  (value 2).
6
7 peaks, _ = find_peaks(x, height=2.5)
8 # peaks: array([1, 3]) -- values 3 and 4.
9
10 peaks, _ = find_peaks(x, height=(2.0, 3.5))
11 # peaks: array([1, 5]) -- values 3 and 2.
```


The threshold Parameter

- ▶ Filters peaks by the vertical drop to each of their two immediate neighbors.
- ▶ For a peak at index p , the left drop is $x[p] - x[p - 1]$ and the right drop is $x[p] - x[p + 1]$.
- ▶ Both drops must independently satisfy the bounds $(t_{\min}, t_{\max})$:

$$t_{\min} \leq x[p] - x[p \pm 1] \leq t_{\max}. \quad (3)$$

- ▶ A large $t_{\min}$ selects only sharp, isolated peaks.
- ▶ The computed drops are stored in `properties['left_thresholds']` and `properties['right_thresholds']`.

The threshold Parameter – Example

Example 2 (threshold filtering)

```
1 x = np.array([0, 2, 1, 3, 0])
2 # Peak at index 1 (value 2):
3 #   left drop = 2 - 0 = 2,   right drop = 2 - 1
      = 1.
4 # Peak at index 3 (value 3):
5 #   left drop = 3 - 1 = 2,   right drop = 3 - 0
      = 3.
6
7 peaks, _ = find_peaks(x, threshold=1.5)
8 # Peak 1: right drop = 1 < 1.5 --> rejected.
9 # Peak 3: both drops >= 1.5 --> accepted.
10 # peaks: array([3]).
```

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

The distance Parameter

- ▶ Requires that any two accepted peaks are separated by at least d samples:

$$|p_i - p_j| \geq d \quad \text{for all } i \neq j. \quad (4)$$

- ▶ When two peaks violate this constraint, the *lower* peak is removed.
- ▶ Prevents the detection of multiple closely spaced local maxima as separate events.
- ▶ This is useful when the sampling rate is high relative to the expected event rate.

The distance Parameter – Example

Example 3 (distance filtering)

```
1 x = np.array([0, 3, 1, 4, 1, 3, 0])
2 # Peaks at indices 1 (value 3), 3 (value 4), 5  
   (value 3).
3
4 peaks, _ = find_peaks(x)
5 # peaks: array([1, 3, 5]).
6
7 peaks, _ = find_peaks(x, distance=3)
8 # Indices 1 and 3 are 2 samples apart: peak 1  
   (lower) removed.
9 # Indices 3 and 5 are 2 samples apart: peak 5  
   (lower) removed.
10 # peaks: array([3]).
```

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

What is Prominence?

- ▶ **Prominence** measures how much a peak stands out from its surrounding signal, independently of its absolute height.
- ▶ For a peak at index p , the prominence is computed as follows:
 1. On each side of p , extend outward until reaching a sample higher than $x[p]$ or the signal boundary.
 2. Find the minimum of x within each resulting interval.
 3. The **base height** is the larger of these two minima.
- ▶ Prominence is then defined as

$$\text{prom}(p) = x[p] - \text{base}(p). \quad (5)$$

Prominence – Intuition

- ▶ A peak sitting on a high plateau has low prominence even if its absolute value is large.
- ▶ A peak rising from a deep valley has high prominence even if its absolute value is moderate.
- ▶ Prominence is therefore robust to long-range baseline drifts in the signal.
- ▶ The concept mirrors topographic prominence used for mountains:
 - ▶ A mountain's prominence is the minimum altitude one must descend to reach any higher summit.

The prominence and wlen Parameters

- ▶ **prominence**: a scalar or pair ($p_{\min}, p_{\max}$) that filters peaks by their prominence.
- ▶ **wlen**: limits the search for the base height to a window of **wlen** samples centred on each peak, making the computation local and faster.
- ▶ Computed prominences are stored in `properties['prominences']`.

Example 4 (prominence filtering)

```
1 x = np.array([0, 1, 0, 5, 1, 4, 0])
2 # Prominences: index 1 -> 1.0, index 3 -> 5.0,  
  index 5 -> 3.0.
3
4 peaks, props = find_peaks(x, prominence=2)
5 # peaks: array([3, 5])
6 # props['prominences']: array([5., 3.])
```

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

What is Width?

- ▶ The **width** of a peak is the horizontal extent (in samples) of the peak at a reference height determined by `rel_height`.
- ▶ The reference height for a peak at index p is

$$h_{\text{ref}}(p) = x[p] - \text{rel_height} \times \text{prom}(p). \quad (6)$$

- ▶ The width is the distance between the two interpolated points where the signal crosses h_{ref} on each side of p .
- ▶ The default `rel_height = 0.5` gives a half-prominence width, analogous to the full width at half maximum (FWHM).

The `width` and `rel_height` Parameters

- ▶ `width`: a scalar or pair ($w_{\min}, w_{\max}$) that filters peaks by their width in samples.
- ▶ `rel_height`: a value in $(0, 1]$ that sets the fractional prominence level at which the width is measured.
- ▶ Computed widths are stored in `properties['widths']`.

Example 5 (`width` and `rel_height`)

```
1 x = np.array([0, 0, 5, 0, 0])
2 # Single sharp peak at index 2, prominence = 5.
3
4 peaks, props = find_peaks(x, width=0,
5     rel_height=0.5)
6 # h_ref = 5 - 0.5*5 = 2.5 --> width ~ 1.0.
7
8 peaks, props = find_peaks(x, width=0,
9     rel_height=0.9)
10 # h_ref = 5 - 0.9*5 = 0.5 --> width ~ 1.8.
```

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

The `plateau_size` Parameter

- ▶ Some peaks have a **flat top**: a consecutive run of equal-valued samples at the peak height.
- ▶ The plateau size is the number of samples in this flat region.
- ▶ A standard sharp peak has plateau size 1.
- ▶ `plateau_size`: a scalar or pair ($s_{\min}$, $s_{\max}$) that filters peaks by the size of their flat top.
- ▶ Computed plateau sizes are stored in `properties['plateau_sizes']`.

Example 6 (`plateau_size` filtering)

```
1 x = np.array([0, 2, 2, 2, 0, 3, 0])
2 # Flat-top peak at indices 1-3 (plateau size 3).
3 # Sharp peak at index 5 (plateau size 1).
4
5 peaks, _ = find_peaks(x, plateau_size=2)
6 # peaks: array([2]) -- only the flat-top peak
   passes.
```

Combining Multiple Parameters

- ▶ All parameters can be combined in a single call.
- ▶ Each parameter acts as an independent filter; a peak must satisfy all conditions simultaneously.

Example 7 (Combined filtering)

```
1 import numpy as np
2 from scipy.signal import find_peaks
3
4 rng = np.random.default_rng(42)
5 t = np.linspace(0, 10 * np.pi, 500)
6 x = np.sin(t) + 0.3 * rng.standard_normal(500)
7
8 peaks, props = find_peaks(
9     x,
10     height=0.5,           # peak must be above 0.5
11     distance=20,          # at least 20 samples apart
12     prominence=0.5,       # must stand out by 0.5
13     width=3               # at least 3 samples wide
14 )
```

Summary of Parameters

Parameter	Description.
height	Bounds on the absolute peak value $x[p]$.
threshold	Bounds on the vertical drop to each immediate neighbor.
distance	Minimum number of samples between any two peaks.
prominence	Bounds on prominence (how much the peak stands out).
wlen	Window size for local prominence and width computation.
width	Bounds on peak width in samples.
rel_height	Fractional prominence level at which width is measured.
plateau_size	Bounds on the size of the flat top of a peak.

Outline

Peaks in Signals

Filtering by Value

Filtering by Distance

Prominence

Width

Plateau Size and Combining Parameters

Exercises

Exercise 1 – Predict the Output

Given the signal:

```
1 x = np.array([0, 1, 0, 3, 2, 4, 0, 2, 0])
```

The local maxima are at indices 1,3,5,7 with values 1,3,4,2 respectively.

- (a) Without running code, which peaks survive `find_peaks(x, height=2.5)`?
- (b) Without running code, which peaks survive `find_peaks(x, threshold=1.5)`?

Verify your answers by running the code in Python.

Exercise 1 – Solution

(a) `height=2.5` keeps peaks with $x[p] \geq 2.5$:

- ▶ Index 1 (value 1): $1 < 2.5$ – rejected.
- ▶ Index 3 (value 3): $3 \geq 2.5$ – accepted.
- ▶ Index 5 (value 4): $4 \geq 2.5$ – accepted.
- ▶ Index 7 (value 2): $2 < 2.5$ – rejected.

Answer: `peaks = array([3, 5])`.

(b) `threshold=1.5` requires both drops ≥ 1.5 :

- ▶ Index 1: drops 1, 1 – rejected (both < 1.5).
- ▶ Index 3: drops 3, 1 – rejected (right drop $1 < 1.5$).
- ▶ Index 5: drops 2, 4 – accepted.
- ▶ Index 7: drops 2, 2 – accepted.

Answer: `peaks = array([5, 7])`.

Exercise 2 – Prominence by Hand

Given the signal:

```
1 x = np.array([0, 4, 1, 3, 0])
```

The peaks are at index 1 (value 4) and index 3 (value 3).

- (a) Compute the prominence of each peak by hand using the definition from the presentation.
- (b) Which peaks survive `find_peaks(x, prominence=2.5)`?
- (c) Verify your answers by running `find_peaks(x, prominence=0)`, which returns all peaks together with `properties['prominences']`.

Exercise 2 – Solution

(a) Prominence of index 1 (value 4):

- ▶ Left boundary: $\min(x[0]) = 0$.
- ▶ No peak to the right exceeds 4, so right boundary: $\min(x[2], x[3], x[4]) = 0$.
- ▶ $\text{base} = \max(0, 0) = 0$, so $\text{prom}(1) = 4 - 0 = 4$.

Prominence of index 3 (value 3):

- ▶ Left: the higher peak at index 1 bounds the window. $\min(x[2]) = 1$.
- ▶ Right boundary: $\min(x[4]) = 0$.
- ▶ $\text{base} = \max(1, 0) = 1$, so $\text{prom}(3) = 3 - 1 = 2$.

(b) **prominence=2.5**: index 1 ($\text{prom} = 4 \geq 2.5$) survives; index 3 ($\text{prom} = 2 < 2.5$) does not. Answer: **peaks = array([1])**.

Exercise 3 – Choose the Right Parameters

The signal below contains two large anomaly peaks at indices 7 and 11 and several smaller noise peaks:

```
1 x = np.array([0, 2, 0, 1, 0, 3, 0, 7, 0, 2, 0, 6, 0, 1, 0])
```

- (a) Write a `find_peaks` call using only the `height` parameter to detect only the anomalies at indices 7 and 11.
- (b) Write a `find_peaks` call using only the `prominence` parameter to detect only the anomalies.
- (c) Verify that both calls return `peaks = array([7, 11])`.

Exercise 3 – Solution

Peak values at indices 1, 3, 5, 7, 9, 11, 13 are 2, 1, 3, 7, 2, 6, 1.

Prominences at the same indices are 2, 1, 3, 7, 2, 6, 1.

The largest noise peak (index 5) has height 3 and prominence 3.

The anomalies (indices 7 and 11) have height 7, 6 and prominence 7, 6.

(a) Any threshold strictly greater than 3 works for **height**:

```
1 peaks, _ = find_peaks(x, height=3.5)
2 # peaks: array([7, 11]).
```

(b) Any threshold strictly greater than 3 works for **prominence**:

```
1 peaks, _ = find_peaks(x, prominence=3.5)
2 # peaks: array([7, 11]).
```

Revision

In this presentation you learned about:

- ▶ What a peak is and why finding peaks matters for anomaly detection.
- ▶ The `scipy.signal.find_peaks` API and its return values.
- ▶ Filtering by absolute peak value (`height`).
- ▶ Filtering by sharpness relative to immediate neighbors (`threshold`).
- ▶ Enforcing a minimum separation between peaks (`distance`).
- ▶ What prominence measures and how to filter by it (`prominence`, `wlen`).
- ▶ How peak width is defined and measured (`width`, `rel_height`).
- ▶ Handling flat-top peaks (`plateau_size`).